ACTIVIDAD PRACTICA

Diseño arquitectural del sistema con IA

Guía de trabajo para alumnos

En esta actividad convertirán el análisis de negocio elaborado en la Unidad 2 en una propuesta de arquitectura de software. Trabajarán con la misma empresa y el mismo caso. La inteligencia artificial podrá apoyar el diseño y la revisión, pero cada decisión deberá justificarse con evidencia del proyecto.

Empresa asignada	CargoBox
Integrantes	Pedro Omar Cardona Alvarez 2226579 Kloudenn Jafed Jiménez Valdivieso 2226639 Leonardo Acosta Morales 2226717
Fecha de entrega	12/10/2026

Cisco Confidential

1 Proposito de la actividad

El objetivo es organizar las reglas y responsabilidades del sistema antes de programarlo por completo. Al terminar, el equipo deberá demostrar cómo pasó de los requerimientos de la Unidad 2 a una arquitectura que separa el dominio, la aplicación y la infraestructura.

La actividad no consiste en pedir a una herramienta de IA que produzca una solución terminada. El trabajo consiste en plantear decisiones, solicitar alternativas, revisar las respuestas y documentar por qué una propuesta sí corresponde o no al negocio analizado.

Resultados de aprendizaje

- Comparar Transaction Script, Table Module y Domain Model con base en la complejidad real del caso.
- Modelar el dominio mediante lenguaje ubicuo, entidades, Value Objects, Aggregates y Domain Events.
- Diseñar una Service Layer que coordine casos de uso sin absorber las reglas del dominio.
- Separar la persistencia mediante Repository y definir el alcance transaccional con Unit of Work.
- Distinguir Commands y Queries en una propuesta básica de CQRS.
- Evaluar la arquitectura con los principios SOLID y registrar las decisiones en ADR.
- Usar IA como copiloto y conservar evidencia de la revisión humana.

2 Continuidad con la Unidad 2

Trabajarán con la misma empresa y no comenzarán un caso nuevo. Antes de diseñar la arquitectura, reúnan los productos elaborados en la Unidad 2 y asegúrense de que todos los integrantes comprenden el proceso principal del negocio.

Insumo de la Unidad 2	Como se utilizara en la Unidad 3
BRD	Delimitar alcance, actores, necesidades, restricciones, supuestos y criterios de exito.
Historias de usuario y Gherkin	Identificar casos de uso, condiciones de negocio y comportamientos esperados.
BPMN	Reconocer la secuencia, decisiones, participantes, excepciones y puntos de coordinacion.
Glosario y modelo de dominio	Mantener un lenguaje ubicuo y localizar conceptos con identidad, valor o ciclo de vida.
Matriz de trazabilidad	Relacionar cada decision arquitectural con una necesidad, historia, regla o escenario.
Repositorio de GitHub	Conservar versiones, diagramas, documentos, prompts y codigo generado o revisado.

Regla de continuidad

Si el equipo detecta una contradicción o un dato faltante en la Unidad 2, deberá registrarlo como hallazgo. Puede ajustar el análisis anterior, pero debe explicar el cambio y actualizar la trazabilidad. No se permite inventar nuevas reglas solo para acomodar una arquitectura.

Cisco Confidential

3 Empresas de trabajo

Cada equipo continuará con la empresa que trabajó en la Unidad 2. La siguiente tabla recuerda el contexto general de los nueve casos. Las preguntas sirven para enfocar la revisión; no indican qué patrón, clases o componentes deben elegir.

Empresa	Contexto	Pregunta de analisis
Mercado Norte	Alimentos, perecederos y promociones	¿Que reglas cambian por producto, sucursal, fecha o existencia?
TecnoNova Retail	Electronica, numeros de serie y garantias	¿Que decisiones requieren conservar identidad y ciclo de vida?
Moda Urbana	Ropa, tallas, colores y temporadas	¿Donde aparecen variantes, reservas, cambios y devoluciones?
HogarMax	Hogar, unidades voluminosas y ventas a credito	¿Que operaciones coordinan inventario, credito y entrega?
SaludPlus Farmacias	Medicamentos, lotes y caducidad	¿Que invariantes deben cumplirse antes de autorizar una operacion?
SportZone	Articulos deportivos y comisiones	¿Que reglas pertenecen a la venta y cuales a procesos posteriores?
Pet and Co	Productos, servicios y suscripciones	¿Que comportamientos dependen del plan, vigencia o tipo de servicio?
Casa and Estilo	Muebles, anticipos y entregas	¿Que estados y eventos cambian durante el cumplimiento de un pedido?
LibroMundo	Libros, ISBN y preventas	¿Que cambia entre una venta normal, una preventa y una cancelacion?

4 Reglas generales

- Mantengan el alcance aprobado en el BRD y seleccionen un proceso principal para el diseño detallado.
- Todas las decisiones deben relacionarse con requerimientos, reglas, historias o escenarios existentes.
- No se evaluará la cantidad de clases. Se evaluará si cada responsabilidad está ubicada y justificada correctamente.
- Los nombres usados en diagramas, código y documentos deben coincidir con el glosario del negocio.
- El código puede ser parcial o boilerplate. Debe permitir revisar la organización y las responsabilidades propuestas.
- Conserven las iteraciones realizadas con IA. No entreguen una respuesta generada sin revisión.

5 Ruta de trabajo

Realicen las etapas en orden. Una etapa puede obligarlos a corregir una decisión anterior; registren el cambio en lugar de ocultarlo.

Cisco Confidential

Etapa	Trabajo	Resultado
1	Preparar el caso	Revisar U2, delimitar proceso y seleccionar casos de uso.
2	Elegir el patron	Comparar alternativas y justificar la opcion elegida.
3	Construir el mapa de dominio	Definir lenguaje, responsabilidades, limites y eventos.
4	Diseñar la Service Layer	Organizar la coordinacion de los casos de uso.
5	Separar persistencia y transacciones	Proponer Repository y Unit of Work.
6	Separar comandos y consultas	Aplicar CQRS basico donde tenga sentido.
7	Revisar con SOLID	Detectar acoplamiento y responsabilidades incorrectas.
8	Representar la arquitectura	Crear UML, secuencias y ADR.
9	Validar y entregar	Trazar, probar, depurar y organizar evidencias.

Cisco Confidential

6 Etapa 1 Preparar el caso

Seleccionen de tres a cinco casos de uso relacionados con el proceso principal. Deben incluir al menos un flujo normal y un flujo con excepción o rechazo. Para cada caso de uso, identifiquen el actor, el objetivo, la información de entrada, las reglas involucradas y el resultado esperado.

ID	Caso de uso (Objetivo)	Actor	Información de entrada	Reglas relacionadas	Resultado esperado
CU-01	**Capturar los datos del envío** (Flujo normal)	Empleado de sucursal	Datos del envío: remitente, destinatario, tipo de servicio y seguro.	Llega un paquete a sucursal y el empleado captura los datos.	El sistema genera la guía y el envío queda como 'recibido'.
CU-02	**Consultar el estado del paquete** (Flujo normal)	Cliente	Número de guía.	El cliente tiene el número de guía y lo ingresa para revisar el sistema.	Se muestra la última actualización del paquete (recibido, en tránsito, etc.).
CU-03	**Confirmar la recepción del paquete** (Flujo normal)	Destinatario	Firma en la pantalla o fotografía.	El repartidor llega con el paquete al destino.	El envío queda marcado como entregado y se guarda la prueba.
CU-04	**Reportar dirección incorrecta** (Flujo de rechazo)	Repartidor	Selección de estado 'dirección incorrecta' en la aplicación.	El repartidor llega a una dirección que está equivocada o no existe.	El paquete regresa al centro de distribución y se avisa a Atención al Cliente.
CU-05	**Reportar ausencia del cliente** (Flujo de excepción)	Repartidor	Selección de estado 'ausente' en la aplicación.	El cliente no se encuentra al momento de la entrega.	El sistema programa un nuevo intento para el siguiente día hábil, si no se han agotado los intentos.

Antes de continuar, comprueben que los casos seleccionados aparecen en las historias de usuario o en el BPMN. Si no existe esa relación, el caso debe justificarse como corrección de alcance.

Cisco Confidential

Justificación de los Casos de Uso Seleccionados

Después de revisar los cinco casos de uso propuestos, confirmamos que todos aparecen en las Historias de Usuario de CargoBox. Por esta razón, no es necesario hacer ninguna corrección de alcance, ya que todos nuestros escenarios vienen directamente de los requerimientos originales del proyecto.

Aquí mostramos cómo se relaciona cada uno:

- El **CU-01 (Capturar los datos del envío)** sale directo de la **US1**, donde el empleado de la sucursal registra la información del paquete.
- El **CU-02 (Consultar el estado del paquete)** viene de la **US5**, que es cuando el cliente revisa las actualizaciones de su envío.
- El **CU-03 (Confirmar la recepción del paquete)** se relaciona con la **US13**, que pide la firma o foto del destinatario para comprobar la entrega.
- El **CU-04 (Reportar dirección incorrecta)** corresponde a la **US9**, que cubre la situación de rechazo cuando el repartidor no encuentra el domicilio o los datos están mal.
- El **CU-05 (Reportar ausencia del cliente)** está basado en la **US10**, que nos dice qué hacer como excepción si el cliente no está en casa al momento de la entrega.

Con esto demostramos que nuestra selección cubre las partes principales del servicio (recibir, rastrear y entregar paquetes) y también los problemas más comunes, cumpliendo exactamente con lo que pide la actividad.

Cisco Confidential

7 Etapa 2 Elegir el patron de logica de negocio

Comparen Transaction Script, Table Module y Domain Model. La decisión deberá basarse en el comportamiento del sistema: cantidad de reglas, interacción entre conceptos, cambios esperados, manejo de estados y necesidad de proteger invariantes.

Trabajo solicitado

- Construyan una tabla comparativa con criterios propios del caso.
- Incluyan ventajas, riesgos y consecuencias de cada alternativa en su empresa.
- Seleccionen un patrón principal. Pueden proponer una combinación solo si delimitan dónde se usa cada patrón.
- Relacionen la decisión con al menos tres evidencias de la Unidad 2.

Criterio del caso	Transaction Script	Table Module	Domain Model	Evidencia U2 (Caso CargoBox)
Complejidad de reglas	Dificulta el mantenimiento; la lógica para revisar si un paquete tiene seguro o cuántos intentos de entrega lleva se duplicaría en varias partes.	Manejo intermedio; agrupa la lógica por tabla, pero se complica al cruzar reglas entre sucursales, envíos y seguros.	**Excelente;** encapsula reglas complejas directamente en las entidades, como calcular automáticamente si aplica un reintento de entrega.	El sistema exige automatizar reglas operativas que antes se hacían por "costumbre", como el límite de 2 intentos de entrega.
Cambios esperados	Frágil; si CargoBox agrega un nuevo tipo de servicio o cambia la política de seguros, habría que reescribir flujos completos.	Depende del esquema de la base de datos; si se agregan nuevas formas de rastreo, el módulo puede volverse rígido.	**Altamente adaptable;** agregar nuevas políticas de envío no afecta a las demás entidades ni a la base de datos.	Se busca mejorar la visibilidad del rastreo, lo que requerirá flexibilidad para agregar nuevos estados o notificaciones futuras.

Cisco Confidential

Criterio del caso	Transaction Script	Table Module	Domain Model	Evidencia U2 (Caso CargoBox)
Interacción entre conceptos	Ineficiente; requeriría consultas repetitivas entre los datos del cliente, el empleado y el paquete.	Utiliza conjuntos de datos para interactuar, limitando un poco la fluidez cuando un repartidor reporta incidencias.	**Navegación natural;** el Repartidor interactúa directamente con el objeto Envío para marcarlo como "ausente" o subir una foto de daño.	El estado de un envío involucra a varios actores: desde el Empleado que genera la guía hasta el Supervisor que revisa el seguro.
Estados e invariantes	Riesgoso; las validaciones de estado (ej. verificar si ya salió de sucursal) dependen de que el programador no olvide poner la condición.	La lógica de estado reside en la base de datos, lo que separa el comportamiento de las reglas del negocio.	**Estricto;** los objetos controlan sus transiciones. Un paquete se protege a sí mismo para no ser cancelado si ya está en tránsito.	Transiciones críticas: un envío "recibido" pasa a "en tránsito", y no puede cancelarse una vez que sale de la sucursal.
Ventajas	Desarrollo inicial rápido si solo se quisiera registrar la entrada y salida de paquetes simples.	Buen equilibrio para desarrollar rápidamente los reportes gerenciales de tiempos de entrega e incidencias.	Máxima protección de datos y representación exacta del ciclo de vida del paquete y sus incidencias en el código.	Es crucial reducir llamadas de clientes automatizando las actualizaciones de estado con precisión.

Cisco Confidential

Criterio del caso	Transaction Script	Table Module	Domain Model	Evidencia U2 (Caso CargoBox)
Riesgos	Alta probabilidad de generar inconsistencias (ej. paquetes entregados que siguen marcados en tránsito).	Limitado ante las excepciones complejas de los repartidores (direcciones incorrectas, daños).	Inversión de tiempo significativa al inicio para diseñar bien las clases de Envío, Cliente y Sucursal.	Existen reglas de negocio estrictas, como avisar si un rastreo se detiene por 24h o 48h.
Consecuencias	Costos de mantenimiento insostenibles y fallas en el rastreo, que es el problema principal de la empresa.	Arquitectura muy acoplada a las tablas, dificultando separar la operación diaria de los reportes.	Requiere implementar un framework (ORM) para conectar los objetos a la base de datos, pero asegura el proyecto a largo plazo.	Construir una plataforma centralizada que evite decisiones por experiencia y controle todo el flujo logístico.

Producto de la etapa: una decisión argumentada de una a dos páginas. No basta escribir que un patrón es más fácil, moderno o escalable. La explicación debe indicar qué problema concreto del caso ayuda a resolver y qué costo introduce.

Cisco Confidential

2. Decisión Argumentada

Patrón seleccionado: Modelo de Dominio (*Domain Model*), apoyado con el Módulo de Tabla (*Table Module*) únicamente para hacer los reportes.

Para cubrir las necesidades de CargoBox, la mejor opción es usar el Modelo de Dominio como la base de nuestro sistema. Elegimos esto no solo porque sea una herramienta avanzada, sino porque ataca el problema principal de la empresa: evitar que las decisiones se tomen por "costumbre" y mejorar la forma en que se rastrean los paquetes.

¿Qué problema concreto nos ayuda a resolver?

Este modelo sirve para proteger las reglas de la empresa y evitar que ocurran errores. Por ejemplo, hay una regla clara: un envío solo se puede cancelar si sigue en la sucursal. En lugar de hacer que el programa revise esto de forma insegura en cada paso, el paquete mismo "sabrá" rechazar la cancelación si ya está en tránsito. También nos ayuda a automatizar procesos; si un cliente no está en casa, el sistema programará solo el siguiente intento, cuidando de no pasarse del límite de dos veces.

¿Qué desventajas o costos tiene?

Usar esta forma de trabajo nos tomará más tiempo al principio y requiere que el equipo la entienda bien. Nos obliga a planear con mucho cuidado cómo se van a relacionar los paquetes, las sucursales y los problemas antes de empezar a programar. Además, necesitaremos usar algunas herramientas extra para poder guardar toda esta información en la base de datos.

Nuestra propuesta para que el sistema sea más rápido:

Aunque el Modelo de Dominio es excelente para controlar los envíos en el día a día, puede ser un poco pesado si solo queremos consultar el historial. Por eso, sugerimos mezclar dos ideas: usar el Modelo de Dominio para todo el movimiento logístico (recibir, rastrear y entregar), pero usar el Módulo de Tabla exclusivamente para sacar los reportes de la gerencia. Así podremos generar esa información mucho más rápido sin hacer lento el sistema principal.

Relación con lo aprendido en la Unidad 2:

1. **Control de estados:** Necesitamos ser muy estrictos para que, si un paquete está dañado, pase a revisión de seguros en automático, en lugar de que alguien lo marque como entregado por error.
2. **Relación entre conceptos:** Como hay muchas personas involucradas (repartidores, atención al cliente y supervisores) actualizando el mismo paquete al mismo tiempo, hacer un código simple no sería suficiente ni seguro.
3. **Facilidad para crecer:** Al separar las reglas de la empresa de la base de datos, si en el futuro CargoBox decide cambiar cómo asigna sus centros de distribución, podrá hacerlo fácilmente sin que el programa deje de funcionar.

Cisco Confidential

8 Etapa 3 Construir el mapa de dominio

Transformen el lenguaje y las reglas de la Unidad 2 en un mapa de responsabilidades. Identifiquen los elementos que el sistema debe reconocer, proteger o comunicar. No conviertan automáticamente cada tabla de datos en una entidad del dominio.

Elementos que deben identificar

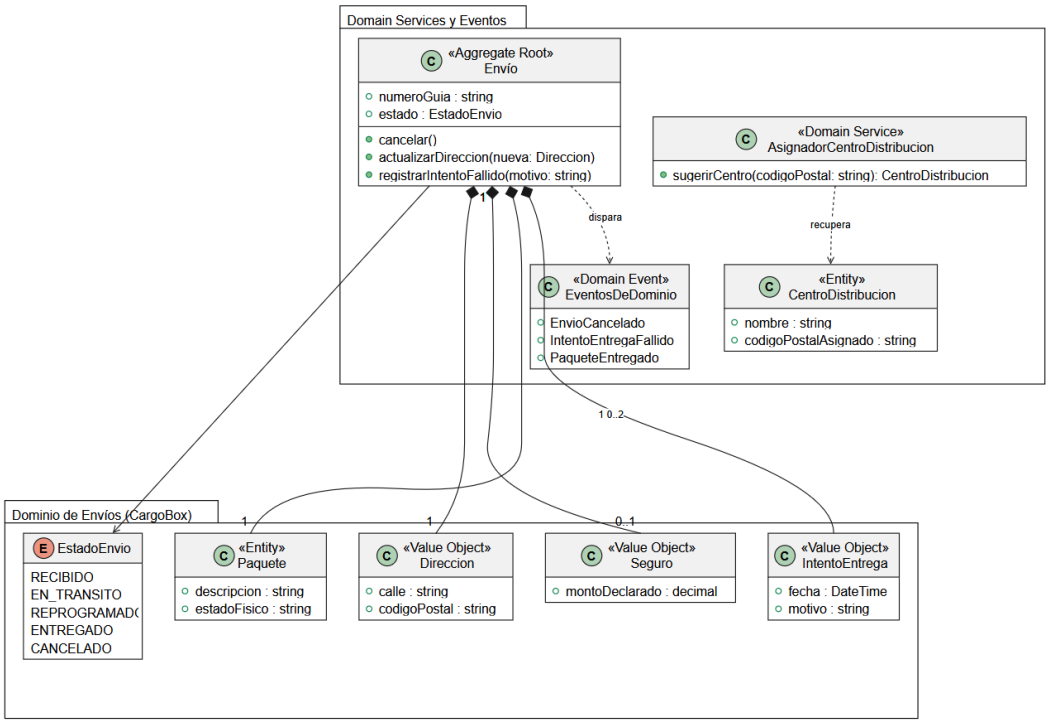
Elemento	Pregunta que debe responder el equipo
Lenguaje ubicuo	Envío, Guía, Paquete, Sucursal, Centro de distribución, Repartidor, Rastreo, Estado, Movimiento, Incidencia (Ausente, Dirección incorrecta), Seguro, Evidencia de entrega
Entidades	Envío: Tiene una identidad única (Número de Guía) y su estado cambia a lo largo del tiempo (recibido, en tránsito, entregado, cancelado) Paquete: Objeto físico vinculado al envío que puede cambiar su estado físico (Buen estado, Dañado).
Value Objects	Dirección: Definida solo por sus datos (calle, número, código postal). Si se corrige, se reemplaza por una nueva Seguro: Definido por el monto declarado; no tiene identidad propia
Aggregates	Agregado Envío (Raíz): Controla el paquete, la dirección destino, el seguro asociado y la lista de intentos de entrega o movimientos.
Domain Services	AsignadorCentroDistribucion: Contiene la lógica para sugerir o asignar el centro correspondiente evaluando el código postal CalculadoraIndemnizacion: Evalúa si un paquete dañado contaba con seguro y determina si procede la indemnización o el reembolso
Domain Events	GuiaGenerada, EnvioCancelado, DireccionActualizada, IntentoEntregaFallido, PaqueteDanado, PaqueteEntregado.

Producto de la etapa

- Diagrama del mapa de dominio con relaciones y límites claramente identificados.
- Glosario actualizado con los términos nuevos o corregidos.
- Ficha de cada Aggregate que indique su raíz, responsabilidades, invariantes y eventos posibles.
- Explicación de las decisiones dudosas o descartadas.

Cisco Confidential

1. Diagrama del Mapa de Dominio



Cisco Confidential

2. Glosario actualizado

- **Aggregate Root (Raíz del Agregado):** El objeto Envío que sirve como única puerta de entrada para modificar los datos de un paquete, garantizando que se cumplan las reglas de negocio.
- **Estado del Envío:** Propiedad que rastrea la fase actual del paquete (Recibido, En Tránsito, En Reparto, Entregado, Cancelado).
- **Invariante:** Regla de negocio estricta que la clase Envío no puede romper bajo ninguna circunstancia (ejemplo: exceder el límite de intentos de entrega).

3. Ficha del Aggregate

- **Raíz del Agregado:** Envío.
- **Responsabilidades:** Mantener la consistencia del ciclo de vida del paquete, validar cambios de estado, y registrar el historial de incidencias e intentos de entrega.
- **Invariantes (Reglas inflexibles a proteger):**
  - No se puede cambiar el estado a Cancelado si el paquete ya salió de la sucursal de origen (US2).
  - No se puede autorizar una DireccionActualizada si el paquete ya llegó a su Centro de Distribución final (US7).
  - El sistema solo permite programar un nuevo intento de entrega al día hábil siguiente si el contador registra menos de 2 intentos previos (US10).
- **Eventos Posibles:**
  - EnvioCreado (Se guardaron datos y se generó guía).
  - AlertaRastreoDetenido (Superó 24h/48h sin actualización).
  - ReprogramacionAutorizada (Cliente ausente con intentos < 2).
  - EvidenciaCapturada (Firma o foto guardada exitosamente).

4. Explicación de decisiones dudosas o descartadas

- **Descarte de Seguro como Entidad:** Se decidió modelar el Seguro como un Value Object en lugar de una Entidad. El negocio solo necesita conocer el monto declarado para decidir si se indemniza el valor o se reembolsa el costo del envío (US4, US12). No requiere un ID independiente ni un ciclo de vida ajeno al Envío.
- **Regla de Costo Extra fuera del Dominio de Envío:** Cuando se superan los 2 intentos de entrega, el paquete regresa a sucursal. La decisión de si se cobra un costo extra al cliente por un tercer intento se descartó del modelo central de operaciones de entrega, ya que pertenece a un dominio de Facturación o Cobranza, por lo que el agregado Envío simplemente finaliza su flujo operativo en "Regresado/No Entregado".

Cisco Confidential

9 Etapa 4 Diseñar la Service Layer

Diseñen los Application Services que expondrán los casos de uso. Cada método debe representar una intención completa del usuario o de otro sistema. El servicio coordina la operación: recibe la solicitud, obtiene los objetos necesarios, invoca el comportamiento del dominio, solicita la persistencia y devuelve un resultado.

La Service Layer no debe convertirse en un lugar donde se acumulen todas las reglas. Si una condición describe lo que el negocio permite, calcula o protege, el equipo debe justificar dónde vive esa regla y por qué.

Servicio	Metodo o caso de uso	Entrada	Colaboradores	Salida	Reglas que delega
EnvioAppService	RecibirPaquete (US1)	Datos, remitente, destinatario, dimensiones, seguro declarado	IEnvioRepository, ICentroDistribucionService	Número de Guía generada	Delega al dominio la creación inicial del objeto, validando que los datos obligatorios no estén vacíos y estableciendo el estado inicial en "Recibido"
EnvioAppService	CancelarEnvio (US2)	Número de Guía, ID de Empleado/Supervisor	IEnvioRepository	Confirmación de cancelación	Delega al Aggregate Envío la regla que dicta que **solo se puede cancelar si el paquete sigue físicamente en la sucursal origen**.
EnvioAppService	ActualizarDireccion (US7)	Número de Guía, Datos de la Nueva Dirección	IEnvioRepository	Notificación de cambio exitoso	Delega al Aggregate Envío la validación de que el paquete **todavía no haya llegado al centro de distribución destino final** antes de aceptar el cambio
EnvioAppService	RegistrarIntentoFallido (US10)	Número de Guía, Motivo (Ausente / Dirección incorrecta)	IEnvioRepository	Estatus (Reprogramado o Devuelto a origen)	Delega al Aggregate Envío la responsabilidad de sumar el intento y validar la regla de negocio: **si van menos de 2 intentos, permite reprogramar al día hábil siguiente**; si van 2 o más, cambia el estado a devolución.

Cisco Confidential

Servicio	Metodo o caso de uso	Entrada	Colaboradores	Salida	Reglas que delega
EnvioAppService	ReportarPaqueteDanado (US11, US12)	Número de Guía, Foto evidencia, Descripción	IEnvioRepository, IAlmacenamientoArchivos	IAlmacenamientoArchivosTicket de caso creado	Delega al dominio la validación de verificar si el envío cuenta con un **seguro declarado** para determinar si amerita una evaluación de indemnización.

Revision obligatoria

¿El nombre del método expresa una acción del negocio?

Sí. En lugar de usar términos técnicos de bases de datos como UpdateData o SaveRecord, se utilizan acciones claras del lenguaje ubicuo del negocio como CancelarEnvio, ActualizarDireccion o RegistrarIntentoFallido.

¿El método coordina o contiene decisiones que deberían estar en el dominio?

El método **solo coordina**. Por ejemplo, el servicio de aplicación no hace la lógica matemática de contar si los intentos de entrega son menores a dos; simplemente busca la Guía mediante el repositorio, le pide a la entidad Envio que registre el intento fallido, y guarda los cambios resultantes. La decisión matemática/lógica vive estrictamente en la entidad.

¿El servicio depende de interfaces o de detalles concretos de base de datos?

Depende estrictamente de interfaces (abstracciones) como IEnvioRepository. El Service Layer no contiene sentencias SQL ni sabe si la base de datos es MySQL o PostgreSQL, lo cual respeta el Principio de Inversión de Dependencias (la "D" de SOLID).

¿La operación tiene un inicio y un final transaccional comprensible?

Sí. Cada método representa una sola transacción (Unit of Work). Inicia al recibir la orden del usuario, procesa los cambios necesarios en memoria y termina al hacer un único *commit* (guardado) en la base de datos. Si algo falla a la mitad, se hace un *rollback* para evitar información corrupta.

¿Los errores del dominio pueden explicarse con términos del negocio?

Sí. Si la capa de dominio rechaza una acción, no arroja un error técnico como "NullReferenceException" o "Error Code 500". En su lugar, arroja excepciones de negocio comprensibles, tales como *"El envío no puede ser cancelado porque ya se encuentra en tránsito"* o *"Se superó el límite de intentos de entrega permitidos"*.

Cisco Confidential

10 Etapa 5 Separar persistencia y transacciones

Definan interfaces de Repository en términos del dominio y describan, por separado, cómo podrían implementarse en infraestructura. Los métodos deben expresar lo que el dominio necesita recuperar o guardar, sin exponer consultas SQL como parte del modelo.

También indiquen qué operaciones deben completarse juntas. La propuesta de Unit of Work debe explicar dónde inicia la transacción, qué cambios incluye y qué ocurriría ante un error.

Repository	Necesidad del dominio	Metodos propuestos	Objeto que recupera o guarda

EnvioRepository	Recuperar el historial del paquete para validar sus reglas (ej. si ya fue entregado o si rebasó intentos) y guardar sus nuevos estados o incidencias.	`ObtenerPorGuia(string numeroGuia)`
		
		`Guardar(Envio envio)`
			Recupera y guarda el **Aggregate Root completo (Envio)**, incluyendo su lista de intentos, dirección actual y seguro.

CentroDistribucionRepository	Asignar o sugerir automáticamente el centro de distribución correcto evaluando el código postal de destino (US8).	ObtenerPorCodigoPostal(string cp)	Recupera la Entidad **`CentroDistribucion`** (Operación estricta de solo lectura).
EvidenciaRepository	Guardar las pruebas físicas de entrega (firma/foto de recepción exitosa) o de daño reportado por el repartidor (US11, US13).	GuardarEvidenciaFisica(string guia, Archivo archivo)	Guarda el **Archivo/Imagen** y recupera la URL o ruta de acceso ligada a la guía.

Alcance Transaccional (Unit of Work)

¿Dónde inicia la transacción?
Arranca en la Capa de Servicio EnvioAppService en el momento exacto en que el repartidor presiona el botón de "Ausente" en su aplicación móvil y el sistema recibe esa instrucción.
¿Qué cambios incluye (operaciones que se completan juntas)?
El sistema agrupa y ejecuta tres pasos seguidos como si fueran uno solo:

- Saca de la base de datos el registro actual de ese envío.
- Aplica la regla de negocio: le suma +1 al contador de intentos y cambia el estado a "Reprogramado" (validando que lleve menos de 2 intentos).
- Guarda la información actualizada y dispara la alerta para Atención al Cliente.

¿Qué ocurriría ante un error (Rollback)? Si hay una falla técnica o se cae el servidor justo antes de terminar de guardar, el *Unit of Work* hace un **Rollback** (da marcha atrás y cancela todo). Esto significa que el paquete se queda con el mismo estado y número de intentos que tenía antes de que el repartidor presionara el botón. Esto te salva de problemas graves, como enviar alertas falsas a Atención al Cliente sobre un intento fallido que en realidad nunca se logró registrar bien en la base de datos.

Cisco Confidential

11 Etapa 6 Separar Commands y Queries

Apliquen CQRS básico para hacer explícita la diferencia entre cambiar el estado y consultar información. No es obligatorio construir bases de datos separadas. El objetivo es que cada operación tenga una responsabilidad clara y que las consultas puedan devolver un modelo apropiado para lectura.

Trabajo solicitado

- Seleccionen al menos dos operaciones que cambien el estado y descríbanlas como Commands.
- Seleccionen al menos dos necesidades de información y descríbanlas como Queries.
- Definan el Handler de cada operación y sus dependencias.
- Diseñen el DTO o View Model de lectura sin reutilizarlo automáticamente como modelo de escritura.
- Justifiquen si alguna parte del caso no necesita esta separación.

Tipo	Nombre	Datos de entrada	Handler	Resultado	Justificacion
Command	RegistrarEnvioCommand	Remitente, destinatario, tipo de servicio, seguro, descripción	RegistrarEnvioHandler	Guía generada y estado *recibido*	Modifica el estado del sistema generando un nuevo agregado Envio en sucursal.
Command	AutorizarCambioDireccionCommand	Número de guía, nueva dirección	AutorizarCambioDireccionHandler	Confirmación de actualización	Valida las reglas de negocio (solo antes de llegar al centro de destino y autorizado por Atención al Cliente).
Query	ConsultarEstadoEnvioQuery	Número de guía	ConsultarEstadoEnvioHandler	DTO con el último estado y movimientos	Permite al cliente consultar su paquete sin sobrecargar ni exponer el modelo de escritura del dominio.
Query	ObtenerReporteIncidenciasQuery	Rango de fechas, tipo de incidencia	ObtenerReporteIncidenciasHandler	Vista de datos (ViewModel) con conteo de incidencias y reclamos	Optimiza la generación de reportes para el gerente de operaciones sin requerir procesamiento manual en Excel.

Cisco Confidential

12 Etapa 7 Revisar la propuesta con SOLID

Revisen la propuesta completa antes de cerrar los diagramas. El análisis debe mostrar decisiones concretas y no una definición memorizada de cada principio.

Principio	Pregunta de revision	Evidencia que deben buscar
S	¿Cada clase o servicio tiene una razon principal para cambiar? Los Application Services (como EnvioService) intentaban validar reglas complejas de negocio directamente.	Responsabilidades mezcladas, clases extensas, servicios que hacen de todo: Riesgo alto de acoplamiento. Se reubicaron las reglas de negocio (como la validación de cancelación en sucursal y los intentos de entrega) dentro del agregado Envio, dejando al servicio únicamente la labor de coordinación.
O	¿Pueden agregar una variante de regla sin editar una cadena creciente de condiciones? La asignación del centro de distribución dependía de condicionales rígidos basados en la costumbre del empleado.	Condicionales que deben modificarse por cada nueva politica: Se introdujo una interfaz de estrategia (AsignadorCentroDistribucion) basada en códigos postales, permitiendo añadir nuevas reglas geográficas sin modificar el código base existente.
L	¿Las implementaciones pueden sustituirse sin cambiar el comportamiento esperado? Las implementaciones concretas de persistencia (SqlEnvioRepository) respetan estrictamente los contratos definidos en las interfaces del dominio.	Subtipos que rechazan operaciones o alteran contratos: Ninguna implementación altera o rechaza inesperadamente los contratos esperados por las capas superiores.
I	¿Cada consumidor depende solo de las operaciones que utiliza? Las consultas de reportes compartían estructuras de datos con los comandos de actualización.	Interfaces grandes con metodos irrelevantes para algunos clientes: Se separaron los modelos de lectura (DTOs) de los modelos de escritura del dominio, evitando que los consumidores dependan de métodos o atributos que no utilizan.
D	¿La aplicacion y el dominio dependen de abstracciones? Las capas de dominio y aplicación interactuaban directamente con detalles de infraestructura y ORMs.	Dependencias directas de SQL, frameworks o servicios externos: Se definieron interfaces de Repository puramente en lenguaje de dominio, invirtiendo la dirección de las dependencias hacia las abstracciones.

Registren al menos cinco hallazgos. Un hallazgo puede confirmar una buena decisión o señalar una corrección necesaria. Para cada uno, incluyan evidencia, impacto y cambio realizado.

Cisco Confidential

Hallazgo 1: Responsabilidad única en la capa de aplicación

- **Evidencia:** En el diseño inicial, el servicio EnvioService contenía directamente la lógica para verificar si un paquete podía cancelarse según su ubicación y los permisos del rol, además de coordinar la transacción.
- **Impacto:** Alto riesgo de acoplamiento y acumulación de lógica de negocio procedural (violando el principio de Responsabilidad Única - S).
- **Cambio realizado:** Se refactorizó el código para mover las reglas de validación (como el estado del paquete en sucursal) al agregado Envio, dejando que el servicio únicamente coordine la llamada.

Hallazgo 2: Extensibilidad en la asignación de centros de distribución

- **Evidencia:** La asignación del centro de distribución dependía de un bloque de condiciones rígidas basadas en la "costumbre" del encargado de la sucursal.
- **Impacto:** Cada vez que se agregaran nuevas reglas geográficas o sucursales, era necesario modificar el código existente, violando el principio de Abierto/Cerrado (O).
- **Cambio realizado:** Se introdujo una interfaz de estrategia denominada AsignadorCentroDistribucion, permitiendo implementar políticas basadas en el código postal de manera independiente.

Hallazgo 3: Sustitución de implementaciones en persistencia

- **Evidencia:** Las clases de los servicios de aplicación interactuaban directamente con detalles específicos de bases de datos relacionales sin mediar contratos formales.
- **Impacto:** Si se deseaba cambiar o mockear la base de datos para pruebas, se rompía el comportamiento esperado, violando el principio de Sustitución de Liskov (L).
- **Cambio realizado:** Se definieron contratos estrictos mediante interfaces de Repository en el dominio (EnvioRepository), asegurando que cualquier implementación secundaria respete exactamente las mismas firmas.

Hallazgo 4: Segregación de contratos de lectura y escritura

- **Evidencia:** Las vistas para consultar el rastreo de paquetes y los reportes gerenciales utilizaban las mismas estructuras de datos y clases que los comandos de actualización.
- **Impacto:** Los clientes y reportes dependían de métodos y atributos innecesarios para su propósito, violando el principio de Segregación de Interfaces (I).
- **Cambio realizado:** Se aplicó CQRS básico separando los DTOs y ViewModels de lectura de los modelos de escritura del dominio, aislando responsabilidades.

Hallazgo 5: Inversión de dependencias en el núcleo del sistema

- **Evidencia:** Las reglas del negocio y el dominio hacían referencias directas a librerías externas de infraestructura y mapeadores de bases de datos.
- **Impacto:** El dominio quedaba fuertemente acoplado a detalles tecnológicos externos, violando el principio de Inversión de Dependencias (D).
- **Cambio realizado:** Se reestructuró el proyecto de modo que el dominio y la aplicación dependan exclusivamente de abstracciones e interfaces, invirtiendo el flujo hacia la capa de infraestructura.

Cisco Confidential

13 Etapa 8 Representar y documentar la arquitectura

Diagrama de clases

El diagrama debe mostrar las clases relevantes para el proceso elegido, sus responsabilidades y relaciones. Diferencien visualmente dominio, aplicación e infraestructura. Incluyan interfaces de Repository, Application Services y los elementos del dominio que hayan justificado.

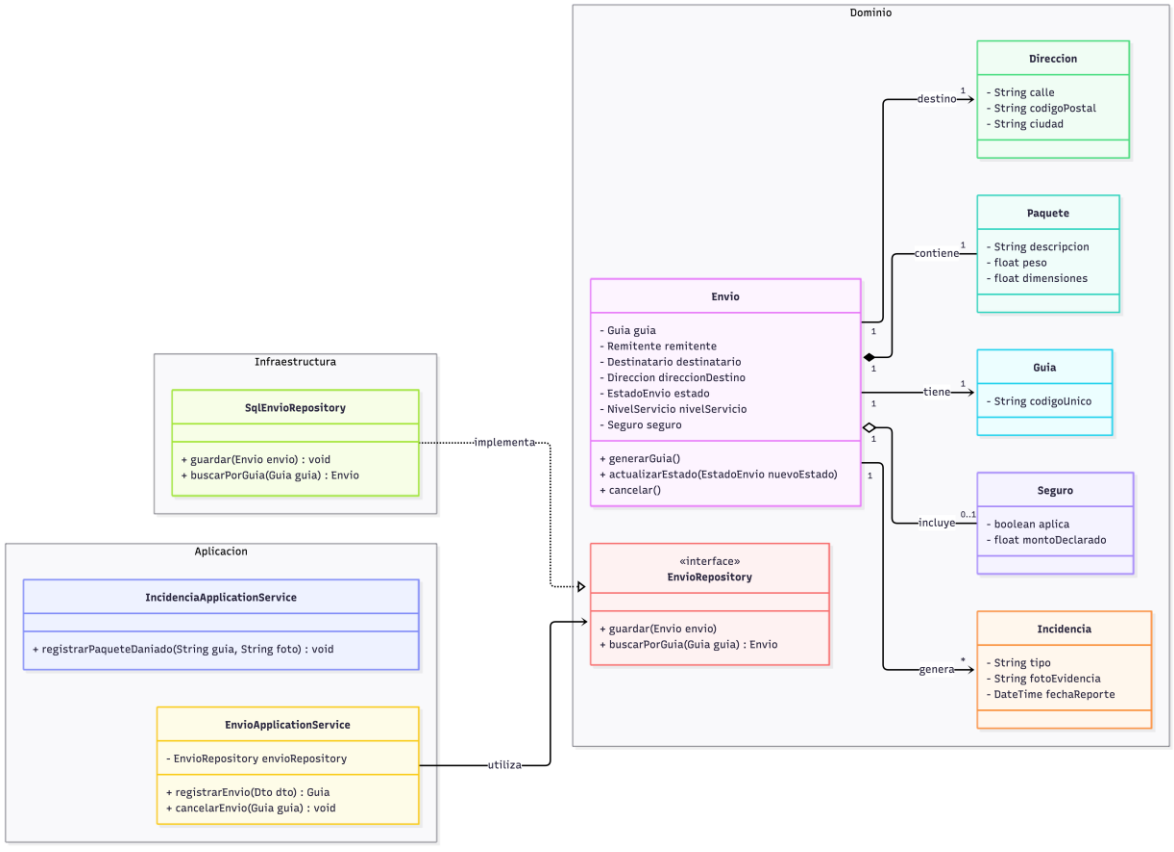
Diagramas de secuencia

Construyan dos diagramas de secuencia. Uno debe representar un flujo principal y el otro un flujo con una regla, excepción o alternativa importante. Los participantes deben corresponder con los componentes del diagrama de clases.

Architecture Decision Records

Elaboren al menos tres ADR. Cada ADR documentará una decisión que tenga consecuencias para el diseño. Como mínimo, registren la selección del patrón de lógica de negocio y dos decisiones adicionales tomadas durante la actividad.

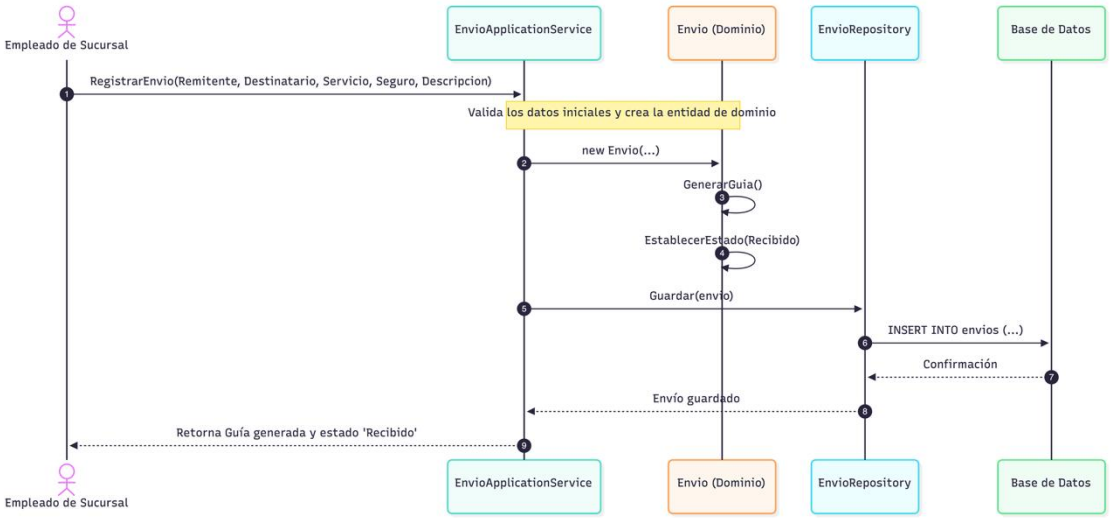
1. Diagrama de Clases



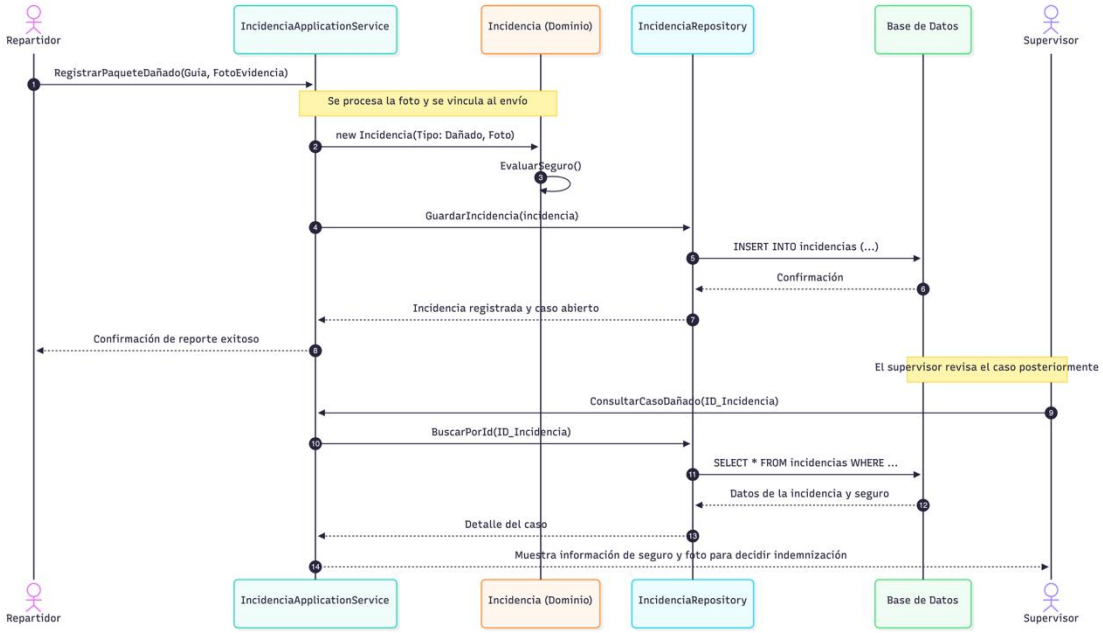
Cisco Confidential

2. Diagramas de Secuencia

- **Flujo Principal:**



- **Flujo con una regla, excepción o alternativa importante:**



Cisco Confidential

3. Architecture Decision Records (ADRs Mínimos)

- **ADR-01: Selección del Patrón de Lógica de Negocio**

Campo del ADR	Contenido que debe redactar el equipo
Titulo	Uso del Patrón *Domain Model* para la gestión de envíos.
Estado	Aceptada.
Contexto	CargoBox maneja reglas complejas de negocio, estados estrictos (recibido, en tránsito, entregado), invariantes de cancelación en sucursal y políticas de seguro.
Alternativas	*Transaction Script* (descartado por acumular lógica procedural) y *Table Module* (descartado por baja cohesión con el dominio).
Decisión	Adoptar *Domain Model* para encapsular el comportamiento y las reglas de negocio dentro de las entidades y agregados.
Consecuencias	Mayor claridad y protección de invariantes, con el costo de una curva de aprendizaje inicial más alta.
Trazabilidad	US1, US2, US7, US10, US11.

- **ADR-02: Implementación de CQRS Básico**

Campo del ADR	Contenido que debe redactar el equipo
Titulo	Separación de operaciones de escritura y lectura en los casos de uso.
Estado	Aceptada.
Contexto	Las consultas de rastreo de clientes y los reportes gerenciales de entregas a tiempo requerían optimización sin comprometer el modelo transaccional.
Alternativas	Mantener una sola estructura de datos tanto para lectura como para escritura.
Decisión	Aplicar CQRS básico separando Commands y Queries con Handlers y ViewModels específicos, sin requerir bases de datos físicas separadas.
Consecuencias	Consultas más rápidas y limpias, y mejor mantenimiento del código.
Trazabilidad	US5, US14, US15.

Cisco Confidential

- **ADR-03: Gestión de Persistencia y Transacciones**

Campo del ADR	Contenido que debe redactar el equipo
Titulo	Uso de Interfaces de Repository y Unit of Work.
Estado	Aceptada.
Contexto	Es necesario garantizar la atomicidad en operaciones complejas como el registro de un envío o la actualización de estados e incidencias.
Alternativas	Ejecutar consultas SQL directamente desde los servicios de aplicación o métodos de dominio.
Decisión	Definir interfaces de Repository orientadas al dominio y gestionar los límites transaccionales mediante un patrón Unit of Work.
Consecuencias	Desacoplamiento total de la infraestructura de base de datos respecto al dominio de CargoBox.
Trazabilidad	US1, US9, US10.

Herramientas permitidas

Pueden utilizar draw.io, PlantUML, Mermaid u otra herramienta que permita entregar diagramas legibles. Los archivos fuente deben incluirse en el repositorio; no entreguen únicamente capturas de pantalla.

Cisco Confidential

14 Vibe Coding con IA

Usarán IA para convertir descripciones del negocio en alternativas de arquitectura, generar estructura inicial, revisar código y producir borradores de diagramas. El equipo sigue siendo responsable de la arquitectura y debe verificar cada resultado contra los documentos de la Unidad 2.

Ciclo de trabajo esperado

Paso	Accion	Evidencia
1	Preparar contexto	Seleccionar fragmentos relevantes del BRD, historias, reglas, BPMN y glosario.
2	Solicitar propuesta	Pedir una alternativa concreta y exigir que la IA declare supuestos.
3	Comparar	Contrastar la respuesta con el caso y con los conceptos de la unidad.
4	Corregir	Modificar nombres, responsabilidades, limites y dependencias que no correspondan.
5	Validar	Revisar trazabilidad, consistencia entre diagramas y comportamiento esperado.
6	Registrar	Guardar prompt, respuesta, decision humana y version resultante.

Prompts de apoyo

Los siguientes formatos son puntos de partida. Completen los corchetes con información de su empresa y eviten enviar datos que no formen parte del caso académico.

- Patrones: Con base en [reglas, cambios y casos de uso], compara Transaction Script, Table Module y Domain Model. Declara tus supuestos y no elijas una opción hasta mostrar sus consecuencias.
- Dominio: Revisa este glosario y estas reglas [contenido]. Propón candidatos a entidades, Value Objects, Aggregates y Domain Events. Para cada candidato, indica la evidencia utilizada y las dudas que requieren validación humana.
- Service Layer: A partir de estos casos de uso [contenido], propone firmas de Application Services. Separa coordinación de reglas de dominio y explica cualquier responsabilidad dudosa.
- SOLID: Analiza este diseño [contenido] y señala posibles violaciones SOLID. No reescribas todo; identifica evidencia, riesgo y alternativas para que el equipo decida.
- Diagramas: Convierte esta arquitectura validada [contenido] en un borrador PlantUML o Mermaid. No agregues clases, reglas o relaciones que no aparezcan en la descripción.

Cisco Confidential

Bitacora obligatoria de IA

Iteracion	Objetivo	Prompt o enlace	Respuesta util	Error o supuesto	Decision del equipo
1	Etapa 1: Casos de Uso	Le pasamos las Historias de Usuario y el diagrama BPMN de CargoBox y le pedimos que nos sacara 5 casos de uso principales y de excepción en una tabla.	Nos hizo bien la lista inicial cubriendo los pasos lógicos, desde que llega el paquete a sucursal hasta que se entrega.	Supuso el formato de la tabla y se le olvidó poner las columnas de "Información de entrada" y "Objetivo" que pedía explícitamente la actividad.	No aceptamos esa primera tabla. Le pedimos que la volviera a hacer agregando los datos que faltaban para cumplir con todo. También nosotros validamos manualmente que el caso de "ausencia" respetara nuestra regla de los 2 intentos.
2	Etapa 2: Patrón de Lógica	Le dijimos que comparara Transaction Script, Table Module y Domain Model usando las reglas de CargoBox (como la de no cancelar paquetes que ya salieron).	Explicó muy bien las ventajas de protección de datos del Domain Model.	Asumió y argumentó que usar Domain Model para absolutamente todo el sistema era lo mejor por ser la arquitectura más "moderna y segura".	Nos dimos cuenta de que usar Domain Model para todo iba a hacer muy lentos los reportes. Decidimos hacer una mezcla por nuestra cuenta: Domain Model para el control diario de los paquetes y Table Module solo para sacar los reportes rápidos de la gerencia.
3	Etapa 3: Mapa de Dominio	Le pasamos nuestro glosario limpio y le pedimos que identificara cuáles eran los Aggregates, Entidades y Value Objects, y que justificara por qué.	Atinó perfecto en que el "Envío" era el Aggregate Root principal del sistema.	Se equivocó en el análisis de prioridades al sugerir que el "Seguro" era una Entidad independiente.	Revisamos el BRD y vimos que a CargoBox solo le importa el "monto del seguro", no rastrearlo por separado. Así que corregimos a la IA y nosotros cambiamos el Seguro a un Value Object para no hacer el diagrama tan complejo.
4	Etapa 4: Service Layer	Con los 5 casos de uso que ya teníamos, le pedimos que diseñara los servicios (Service Layer) para coordinar todo el trabajo.	Nos dio muy buenos nombres para los servicios (Application Services).	Metió las reglas de negocio ahí mismo en el servicio (por ejemplo, puso la lógica para contar los intentos de entrega directo en el Application Service).	Esto rompía la regla de separar responsabilidades. Corregimos el diseño para obligar a que el servicio solo coordine y que la regla matemática de contar los intentos se moviera directamente adentro del objeto "Envío".
5	Etapa 5: Transacciones	Le pedimos que diseñara los repositorios y que nos dijera exactamente dónde empezaba y terminaba una transacción (Unit of Work) en nuestro sistema.	Comprendió bien la idea de agrupar los cambios para que no quedaran datos a medias.	Nos dio una propuesta muy técnica con código SQL revuelto con las reglas de negocio y un manejo de errores súper abstracto.	Quitamos el SQL porque el dominio no debe saber de bases de datos. Nosotros aterrizamos la transacción a la vida real: inicia cuando el repartidor le pica a "Ausente" en su celular; si el internet falla, se hace un Rollback.

Cisco Confidential

Iteracion	Objetivo	Prompt o enlace	Respuesta util	Error o supuesto	Decision del equipo
6	Etapa 6: CQRS	Le pedimos aplicar CQRS para separar la parte de escribir/actualizar estados de la parte de consultar el rastreo de los paquetes.	Explicó cómo la separación ayudaría a los clientes a consultar sus paquetes más rápido.	Nos aconsejó usar dos bases de datos físicas distintas (una para leer y otra para guardar) para que el sistema fuera súper rápido.	Nos pareció una exageración y muy costoso para el tamaño de esta empresa. Decidimos usar un CQRS básico: usamos una sola base de datos, pero en el código sí separamos las vistas de lectura de los modelos de escritura.
7	Etapa 7: Principios SOLID	Le dimos todo lo que llevábamos diseñado y le pedimos que buscara errores según los principios SOLID y nos propusiera soluciones.	Encontró problemas de acoplamiento reales en el código base que no habíamos notado.	El problema fue que nos dio ejemplos de tiendas en línea para solucionarlo, en lugar de usar el lenguaje de paquetería del proyecto.	Tomamos la idea pero la adaptamos a CargoBox. Para el principio Abierto/Cerrado, nosotros diseñamos la interfaz AsignadorCentroDistribucion para quitar código rígido y poder asignar rutas por código postal fácilmente.
8	Etapa 8: Diagramas y ADR	Le pedimos que nos generara el código base en PlantUML para el diagrama de clases y los de secuencia basándose en las etapas anteriores.	Generó la sintaxis correcta de PlantUML, lo cual nos ahorró mucho tiempo de escritura.	El código que generó estaba todo amontonado; visualmente puso la base de datos (infraestructura) revuelta con la aplicación.	Editamos el código UML a mano para separar visualmente bien las 3 capas (Aplicación, Dominio, Infraestructura). También revisamos los ADRs para que no sonaran teóricos, sino con las justificaciones reales que platicamos.
9	Etapa 9: Validación Final	Le pedimos que respondiera las preguntas finales de validación de la guía, checando que todo cuadrara con nuestro BRD de la Unidad 2.	Las respuestas estaban técnicamente correctas y trazaban bien todo con nuestros documentos.	Usó un lenguaje demasiado robótico, técnico y con párrafos larguísimos que no sonaban a nosotros.	Reescribimos todas las justificaciones con nuestras propias palabras para que sonaran naturales y fáciles de explicar, asegurándonos de poder defender en clase cómo nuestro diseño protege las reglas de CargoBox.

Lo que aprendimos al usar la IA como equipo:

- **Facilidad para la redacción de trabajos largos:** Nos dimos cuenta de que la IA es una herramienta increíble para quitarnos de encima el trabajo más pesado. En entregables tan largos como este, la IA nos ahorró muchísimas horas al momento de estructurar tablas inmensas, armar el esqueleto de los documentos y generar la base de textos extensos.
- **Su gran precisión:** Nos sorprendió lo exacta que puede ser la herramienta cuando le pasas la información correcta. Al momento de pedirle que mapeara nuestros casos de uso o que buscara dónde encajaban los principios SOLID en nuestro código, la IA tuvo una precisión excelente para detectar los detalles que le pedimos sin perder el hilo.
- **Redacta como un libro (Es necesaria una intervención humana):** Nuestra mayor queja es que la IA escribe de una forma muy seca como si fuera un libro. Para obtener el resultado deseado es necesario que tenga intervención humana para poder corregir la forma en la cual se redacta la información y así se pueda utilizar de mejor manera en el documento que estamos trabajando

Cisco Confidential

15 Etapa 9 Validar la propuesta

Antes de entregar, recorran cada caso de uso sobre los diagramas y el diseño. El equipo debe poder explicar qué componente recibe la solicitud, dónde se valida cada regla, qué objetos cambian, cómo se persisten los cambios y qué resultado se devuelve.

Validacion	Pregunta	Resultado del equipo
Trazabilidad	¿Cada componente importante responde a una necesidad, regla o caso de uso?	Sí. Cada parte del sistema nace directamente de nuestros Casos de Uso (CU-01 al CU-05) y todo está documentado.
Lenguaje	¿Los mismos terminos significan lo mismo en BRD, UML, codigo y ADR?	Sí. Usamos las mismas palabras clave (como "Envío" o "Guía") tanto en el código como en los diagramas y el glosario.
Responsabilidad	¿Las reglas permanecen en el dominio o en el patron elegido con una justificacion?	Sí. Las reglas estrictas las protege el objeto "Envío". Los servicios solo coordinan el proceso, no toman decisiones.
Consistencia	¿Los diagramas de clase y secuencia representan la misma arquitectura?	Sí. El diagrama de secuencia y el de clases coinciden perfectamente y usan exactamente la misma estructura.
Persistencia	¿El dominio evita depender directamente de SQL, ORM o detalles de infraestructura?	Sí. Separamos la base de datos de las reglas del negocio. Todo lo técnico se aisló en la capa de Infraestructura.
Transaccion	¿El limite de la operacion y su manejo de error estan definidos?	Sí. Definimos que si el sistema falla a la mitad de una operación, se cancela todo en automático para no dejar datos corruptos o a medias.
IA	¿Cada aporte de IA fue revisado y su uso esta registrado?	Sí. Anotamos los prompts del vibecoding en la bitácora y corregimos las sugerencias genéricas para asegurar que el diseño final fuera 100% nuestro.

Cisco Confidential

16 Entregables

Entregable	Contenido minimo
01 Informe de arquitectura	Documento PDF con portada, alcance, comparación de patrones, mapa de dominio, Service Layer, Repository, Unit of Work, CQRS, revisión SOLID, ADR y conclusiones.
02 Diagrama de dominio y clases	Archivo fuente y exportación legible en PDF o PNG.
03 Diagramas de secuencia	Dos archivos fuente y sus exportaciones.
04 Matriz de trazabilidad	Relación entre necesidades, reglas, historias, casos de uso, componentes y decisiones.
05 Bitacora de IA	Prompts, respuestas relevantes, errores detectados, correcciones y decisiones humanas.
06 Codigo base	Estructura de carpetas y código mínimo que represente la arquitectura propuesta.
07 Repositorio GitHub	Todos los archivos organizados, historial de commits y README con instrucciones de revisión.
08 Presentacion y defensa	Exposición breve donde todos los integrantes puedan explicar y defender decisiones.

Estructura sugerida del repositorio

- docs Informe, ADR, trazabilidad y bitacora de IA
- diagrams Archivos fuente y exportaciones
- src Dominio, aplicacion e infraestructura
- tests Escenarios o pruebas seleccionadas
- README Empresa, alcance, instrucciones y enlaces a entregables

17 Rubrica de evaluacion

Criterio	Evidencia esperada	Puntos
Continuidad y trazabilidad	Usa los productos de U2, conserva el alcance y relaciona decisiones con evidencia.	15
Seleccion del patron	Compara alternativas y justifica la eleccion con criterios concretos del caso.	15
Mapa de dominio	Distingue responsabilidades, limites, lenguaje, invariantes y eventos de forma coherente.	20
Service Layer y persistencia	Coordina casos de uso, separa reglas y define Repository y Unit of Work.	15
CQRS y SOLID	Separa operaciones con criterio y revisa el diseño mediante evidencia concreta.	10

Cisco Confidential

Criterio	Evidencia esperada	Puntos
UML y ADR	Los diagramas son consistentes y las decisiones registran contexto, alternativas y consecuencias.	15
Uso responsable de IA	Presenta iteraciones, detecta errores y explica las decisiones humanas.	5
Repositorio y defensa	Entrega organizada, legible y defendida por todos los integrantes.	5

Total 100 puntos

18 Criterios de integridad academica

- La IA puede proponer, revisar y explicar, pero no reemplaza la justificación del equipo.
- Toda afirmación sobre el negocio debe provenir del caso o identificarse como supuesto pendiente de validación.
- El equipo debe conservar la autoría de sus decisiones y poder explicarlas sin leer una respuesta generada.
- El código, los diagramas o los textos generados deben revisarse antes de integrarlos al repositorio.
- Copiar la arquitectura de otro equipo o entregar contenido que no corresponde a la empresa asignada se considera incumplimiento.

19 Lista final de comprobacion

- [ ] Trabajamos con la misma empresa y el mismo alcance de la Unidad 2.
- [ ] La selección del patrón está respaldada por evidencia del caso.
- [ ] El mapa de dominio usa el lenguaje ubicuo del proyecto.
- [ ] Cada Application Service representa un caso de uso y coordina sin concentrar todas las reglas.
- [ ] Las interfaces de Repository hablan en términos del dominio.
- [ ] Los Commands y Queries tienen responsabilidades distintas y justificadas.
- [ ] Revisamos el diseño con SOLID y corregimos los hallazgos relevantes.
- [ ] Los diagramas de clase y secuencia son consistentes.
- [ ] Incluimos al menos tres ADR y una matriz de trazabilidad actualizada.
- [ ] La bitácora demuestra cómo usamos y validamos la IA.
- [ ] El repositorio contiene archivos fuente, exportaciones y un README claro.
- [ ] Todos los integrantes pueden defender las decisiones tomadas.

20 Preguntas para la defensa

- ¿Qué evidencia del negocio influyó más en la selección del patrón?
- ¿Qué responsabilidad fue la más difícil de ubicar y cómo tomaron la decisión?
- ¿Qué regla protege el diseño y qué ocurriría si estuviera en una capa incorrecta?
- ¿Qué cambió después de la revisión con SOLID?
- ¿Qué propuesta de la IA rechazaron o corrigieron y por qué?
- ¿Qué consecuencia negativa aceptaron al tomar su decisión arquitectural principal?

Cisco Confidential